Knowledge-Based Systems 250 (2022) 109166

Contents lists available at ScienceDirect

Knowledge-Based Systems

journal homepage: www.elsevier.com/locate/knosys

Meta-learning based spatial-temporal graph attention network for traffic signal control

Min Wang [a,b], Libing Wu [a,b,*], Man Li [c], Dan Wu [d], Xiaochuan Shi [a,e,*], Chao Ma [e]

[a] School of Computer Science, Wuhan University, Wuhan 430072, China
[b] State Key Laboratory of Integrated Services Networks, Xidian University, Xi'an 710071, China
[c] School of Information Technology, Deakin University, Geelong, VIC, Australia
[d] School of Computer Science, University of Windsor, Windsor N9B 3P4, Canada
[e] School of Cyber Science and Engineering, Wuhan University, Wuhan 430072, China

ARTICLE INFO

Article history:
Received 28 March 2022
Received in revised form 26 May 2022
Accepted 27 May 2022
Available online 3 June 2022

Keywords:
Traffic signal control
Reinforcement learning
Spatial–temporal modeling
Graph attention network
Deep meta-learning
Traffic congestion

ABSTRACT

Traffic signal control is of great importance to the urban transportation systems and public travel, yet it becomes challenging because of two essential factors. First, spatial–temporal correlations are crucial to an intersection scenario. However, existing works have either considered only one of these features or a simple fusion of spatial and temporal information without adequately exploiting the potential correlations. Second, some works using graph neural network treats static graph nodes among adjacent intersections, ignoring the fact that intersection traffic is changing dynamically. These dynamically changing characteristics of an intersection are likewise significant for traffic signal prediction. If these problems are not resolved, the traffic pressure will increase and people's time will be wasted.

To resolve these challenges, we put forward a novel meta-learning spatial–temporal graph attention network (MetaSTGAT) for adaptive traffic signal control. Specifically, we design a graph neural framework with a graph attention network (GAT) and long short-term memory (LSTM) network to obtain spatial and temporal information. The spatial–temporal features are elaborately merged to improve its performance. Besides, to adapt the graph network to the dynamic traffic flow, i.e., the dynamics of the nodes, we propose a meta-learning method for graph neural network's weights generation. This dynamic weight generation process captures the dynamic changes of graph nodes, and thus the dynamic changes of intersections. In this way, the neighboring nodes in the graph network get new weights in advance by additional features when they influence each other. Comprehensive experiments performed in the multi-intersection scenario on synthetic and real-world datasets demonstrate the effectiveness of MetaSTGAT against other state-of-the-art methods. Our method reduces travel time by 12.23%, 19.30%, 13.84%, 10.91%, 8.24%, and 8.74% over the graph-level method CoLight on four synthetic datasets and two real-world datasets, respectively.

1. Introduction

Intelligent signal control is essential for urban traffic systems and is closely related to our daily travel. A feasible traffic signal policy can maximize the traffic flow at intersections, mitigate traffic congestion, and save travel time. Some traditional methods are simple and easy to implement and handle in most traffic scenarios, such as fixed time, real-time traffic systems [1] and the self-organizing traffic light [2]. However, they tend to be inefficient when more vehicles are involved, or the traffic situation becomes complex, especially in scenarios where multiple intersections are connected.

Traditional heuristic methods rely excessively on human-designed rules and pre-defined traffic flow models and thus cannot be applied to complex traffic scenarios. Researchers have recently started to employ deep reinforcement learning (DRL) to address traffic signal control and have made positive achievements [3,4]. These RL methods [5–7] constantly interact with the environment and then directly learn policies for traffic signal control. Nevertheless, there are still many challenges in the signal control problem. In a multi-intersection scenario, controlling the traffic signal cooperatively and fusing the spatial–temporal information properly is very challenging. While most current studies are applicable to multi-intersection scenarios, spatial–temporal information fusion is relatively less discussed in signal control

* Corresponding authors at: School of Computer Science, Wuhan University, Wuhan 430072, China.
E-mail addresses: min.whu@gmail.com (M. Wang), wu@whu.edu.cn (L. Wu), amanda.li@deakin.edu.au (M. Li), danwu@uwindsor.ca (D. Wu), shixiaochuan@whu.edu.cn (X. Shi), chaoma@whu.edu.cn (C. Ma).

https://doi.org/10.1016/j.knosys.2022.109166
0950-7051/

problems. Meanwhile, graph neural networks (GNN) for obtaining traffic states of neighboring intersections are more suitable for graph networks with static nodes but not the dynamic traffic flow. We will propose a better signal control policy to improve traffic efficiency if these two challenges are addressed.

In the multi-intersection scenario, the most typical practice of signal control is to assign each intersection to a separate agent. Then each agent controls the intersection independently, or the adjacent agents can achieve cooperative control by sharing state information [8–10]. In other words, each target intersection can acquire the state of its neighboring intersections and treat it as part of its state s. This way helps agents expand the observation area to receive more comprehensive traffic state information to better control the intersections. However, these methods [8–12] directly concatenate the neighboring states with the target intersection's state and do not consider the time-varying characteristics of traffic flow. CoLight [13] address cooperative control for large-scale multi-intersection scenarios with a graph attention network (GAT) [14]. Unlike the previous method, CoLight takes a graph network to dynamically capture the spatial impact of neighboring intersections. We all know that graph networks have the advantage of dealing with graph-structured data or non-euclidean space data, but they cannot handle the characteristics in the time dimension. Lately, more and more works have used spatial–temporal graph attention networks in travel time estimation [15], traffic flow prediction [16], and signal control problems like our previously proposed DynSTGAT [17]. Nevertheless, most existing methods only fuse the features simply and cannot effectively make use of the potential relations of spatial–temporal information based on cooperative strategy. To address this problem, we first put forward a new spatial–temporal graph attention network (STGAT) consisting of GAT and LSTM to capture the spatial–temporal features of traffic flow. Among them, the GAT network is carefully designed and consists of a multi-head attention mechanism while connecting with the LSTM network. Unlike methods that directly combine spatial and temporal information, our STGAT aims to fully exploit the potential correlations by incorporating neighboring states.

As mentioned before, since the traffic flow is dynamic and the state of each intersection is changing every second, traffic signal control becomes challenging. Although some graph networks such as graph convolutional network (GCN) [18] and graph attention network [14] have recently been widely used and have achieved remarkable results in representation learning [19] and recommendation systems [20]. These graph networks are only applicable to data with static graph nodes, i.e., no node attributes or features change. As a result, many works on dynamic graphs [21,22] have emerged. For example, DyRep [23] is a graph network that adopts representation learning to link two types of graph states, topological changes and node activities. Moreover, ST-MetaNet [24,25] is proposed to learn complex and dynamic spatial–temporal correlations in urban traffic prediction. These correlations differ by location and are related to the surrounding geographic information such as road networks and points of interest. Despite all this, no one has yet tackled traffic signal control from the dynamic graph perspective. Inspired by ST-MetaNet, an intersection's spatial–temporal variation is mainly the total number of vehicles at the intersection and the number of vehicles on each road. In this paper, a meta-learning approach is used to enable the graph network to capture the dynamically changing characteristics of the intersections. Specifically, based on our previously designed STGAT model, we utilize a meta-learning approach to update the weights in the GAT and LSTM networks to adapt it to the dynamic changes of the intersections. The weights are updated based on additional features generated, such as the number of vehicles on the road connecting the two intersections, which are dynamically changing. As we know, the original GAT network [14] has fixed weights when processing graph data, which is obviously not suitable for handling dynamic traffic flow. These are the solutions to the second challenge of traffic signal control.

In summary, we have made the following contributions:

- We first devise a novel and basic spatial–temporal graph neural network entitled STGAT to control traffic signals at the multi-intersection based on reinforcement learning. In the STGAT model, there is a multi-head attention mechanism in GAT network to capture spatial features and an LSTM network to acquire temporal features. Consequently, STGAT can capture the inherent spatial–temporal joint relations of connected intersections.
- We then put forward a new meta-learning based model MetaSTGAT by improving the basic STGAT to deal with dynamically changing intersections. Specifically, we propose a meta graph attention network to model spatial correlations and a meta long short-term memory to capture temporal correlations. Here meta-learning dynamically generates weights in the neural network.
- We perform comparative experiments on synthetic and real-world datasets, demonstrating that the proposed MetaSTGAT largely outperforms traditional and graph-based methods in travel time and throughput evaluation.

We arrange the rest of this paper as follows. Section 2 reviews the relevant research on traffic signal control, graph attention network, and deep meta-learning. Section 3 introduces the problem definition, including the standard intersection and RL environment setting. The proposed spatial–temporal graph attention network and meta-learning approach are shown in Section 4. Experiments and performance evaluation are presented in Section 5. Finally, we draw conclusions and some outlooks in Section 6.

2. Related work

We review three aspects of the research work in this section: traffic signal control based on reinforcement learning, graph attention network, and deep meta-learning.

2.1. Traffic signal control

The traffic signal control (TSC) task aims to maximize the traffic flow or reduce the waiting time by switching the signals at the intersection. We classify TSC into two types of single intersection and multi-intersection scenarios [3]. This paper concentrates on cooperative control in the case of multi-intersection from the perspective of a dynamic graph. The most prevalent approaches are based on reinforcement learning, which typically models intersections and then designs the model's inputs and outputs. In the multi-intersection scenario, a prominent problem is how to achieve cooperative control of these intersections. These methods [8–12] incorporate the state of neighboring intersections by expanding the observation scope. This approach treats the neighboring intersections equally, but in reality, each neighboring intersection impacts the target intersection differently. CoLight [13] first achieves large-scale traffic signal cooperative control with a graph attention network to facilitate communication among intersections. Although CoLight works very well, it only considers the spatial features of the intersection. In our published paper DynSTGAT [17], we improve CoLight from the perspective of spatial–temporal features modeling and fusing. The above two methods are very effective, but they employ graph networks to treat the intersection scenario as a static graph. To solve this problem, we employ meta-learning to dynamically learn the weights in the graph neural network to adapt to changes in dynamic nodes, i.e., the dynamic traffic flow at intersections.

2.2. Graph attention network

Graph neural networks are connectivity models that obtain graph dependencies through information transfer between graph nodes. In recent years, graph neural networks (GNN), especially GCN and GAT, have achieved outstanding results in the field of unstructured data such as text and image [26]. In addition, they are also widely adopted in recommendation systems, representation learning, social networks, and other fields. The emergence of GNN has motivated many applications of spatial–temporal graph neural networks (STGNN) such as traffic prediction [16,27,28]. Compared with GNN, STGNN incorporates both spatial and temporal information. It is common to capture spatial information with a GAT or GCN and temporal information with a recurrent neural network or LSTM. However, most previous works fuse spatial–temporal features independently, neglecting their intrinsic relations. CoLight [13] and GPlight [29] have proven that GNN is very effective for traffic signal control, but this practice is not good enough. Unlike other existing works, the graph attention mechanism we employ is carefully designed to capture spatial–temporal information fully. Therefore, we transform the *key*, *query*, and *value* values in the computation of attention scores, unlike the original GAT network. In other words, we put forward a spatial–temporal graph attention network framework with an attention mechanism to fully exploit the joint spatial–temporal relations. Furthermore, we optimize node weight generation in STGAT to deal with the dynamic traffic flow at the intersection.

2.3. Deep meta-learning

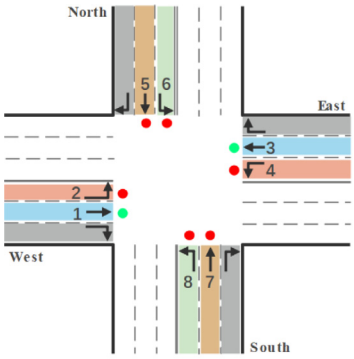
Meta-learning, popularly known as learning to learn, utilizes previously learned knowledge and experience to guide the learning of new tasks to enable the neural network to learn how to learn [30]. Network weights generation is one of the most applied approaches in meta-learning. [31] constructs the learner as a second deep network called *learnet* to learn the model's parameters for one-shot learning. [32] adopts a shared meta-network to capture meta-knowledge in NLP and generates parameters of the task-specific semantic models. [33] proposes the Graph HyperNetwork (GHN) to predict the network's weights and amortize the neural search cost. More studies on other types of meta-learning related to the graph structure. [34,35] build a meta-learner to learn the relations of tasks and then propagate dynamic messages on graph nodes for few-shot learning. Unlike the above methods, in this work, we use meta-learning to primarily solve the problem of regenerating the weights of the graph during the dynamic changes of the nodes. We are the first to employ meta-learning from the perspective of dynamic graphs to solve the network weights generation problem and apply it to traffic signal control.

3. Problem definition

This section presents the definition of the standard intersection and formalizes the TSC problem, then introduces the concepts and settings of reinforcement learning.

3.1. Preliminaries

This paper studies the cooperative control of traffic signals in multi-intersection scenarios based on graph neural networks. Typically, a multi-intersection scene is formed by connecting more than one single intersection. We take Fig. 1 as an example to illustrate the definition of a single intersection, including phases, signals, and lanes.



(a) Intersection

Phase	Lane
A	1&3
B	1&2
C	3&4
D	2&4
E	5&7
F	5&6
G	7&8
H	6&8

(b) Eight phases

Fig. 1. One standard intersection with 8 signal phases.

Entering way. As shown in Fig. 1(a), a standard intersection includes four entering ways: east, south, west, and north. And each entering way has three lanes: a left-turning lane, a straight lane, and a right-turning lane. Except for the rightmost lane, the other two lanes are equipped with traffic lights.

Traffic movement. A traffic movement is a direction in which a vehicle travels along a road, namely left-turning, straight, and right-turning. Suppose a vehicle travels from lane l to lane m, then we call that the traffic movement is (l, m). There are 12 traffic movements[1] in Fig. 1(a), some configured with traffic lights while the rightmost one is not. Moreover, a traffic movement may occupy multiple lanes, but this does not affect our experiments because traffic signals control the traffic movement and not lanes.

Movement signal. Each traffic movement equipped with traffic light are marked with a number, as shown in Fig. 1(a). We use one bit to denote the state of the traffic light: 1 for 'green' and 0 for 'red'.

Signal phase. We use an eight-bit movement signal to define the signal phase p. As illustrated in Fig. 1(b), 8 signal phases are set to control the traffic flow. The phase 'A' is triggered in Fig. 1(a), and it is denoted as $\vec{\mathbf{p}} = [1,0,0,0,0,0,0,0]$. One bit of the $\vec{\mathbf{p}}$ vector denotes a movement signal. If the 'A' phase is triggered, then the east–west traffic is allowed to pass, and the other direction traffic is in the red light phase.

Intersection pressure. Intersection pressure P_i stands for the difference between vehicles entering intersection and the number of vehicles exiting intersection. As shown in Fig. 2, the central intersection pressure is 6, where the total number of vehicles on the entering lane is 13, and the total number of vehicles on the exiting lane is 7.

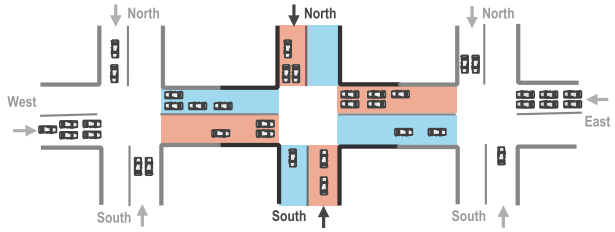


Fig. 2. Traffic pressure diagram of the central intersection.

[1] Only 12 traffic movements are considered in this work, but this method can still be applied to more than 12 or more intersection scenarios.

3.2. Reinforcement learning environment

We use reinforcement learning to model the traffic scenarios, taking traffic states as input to the model to optimize traffic flow at intersections by predicting the action of signals. In this process, the environment represents the intersection, and the agent represents the designed model. Through "trial and error" learning, the agent guides behavior through the rewards obtained from the environment and maximizes the rewards of the model. We formulate the signal control process as a classic Markov Decision Process $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, r, \gamma \rangle$ as follows:

- State $\mathcal{S}$. The intersection subscribes the traffic observation from the simulator, and we define the traffic state s_i^t as a part of the observation. State of intersection $\mathcal{I}_i$ composes of the number of vehicles $\vec{\mathbf{n}}$ on each lane and the current signal phase $\vec{\mathbf{p}}$, $s_i^t = [\vec{\mathbf{n}}, \vec{\mathbf{p}}]$. The length of vector $\vec{\mathbf{n}}$ is the number of intersection lanes or traffic movements, for a standard intersection, length $\vec{\mathbf{n}}$ is 12.
- Action $\mathcal{A}$. The action indicates the permissible combinations of traffic signal that allow vehicles to pass safely. At timestamp t, agent takes one phase p as its action a_i^t. We fix the execution duration of each action to 10 s ($\Delta t = 10$ s), which is the duration of the green light. Additionally, after the duration of the green light, we need to set the yellow light duration ($t_y = 3$ s) for the remaining vehicles to cross the intersection.
- Transition probability $\mathcal{P}$. The state transfer probability describes the dynamic characteristic of the environment. $\mathcal{P}(s_i^{t+1} \mid s_i^t, a_i^t)$ represents the probability when intersection jumps from one state $s_i{}^t$ to the next state $s_i{}^{t+1}$.
- Reward r. Reward indicates the returns received from the environment after the agent takes action and the agent's goal is to maximize the total cumulative reward. In Fig. 2, we define the intersection pressure, the difference between the total number of vehicles entering the lane and the number of vehicles leaving the lane. The intersection pressure Pi coincides with the goal we want to optimize. Therefore, the model's reward is defined as,

$$r_i = -P_i \tag{1}$$

  Based on the size of the reward received from the environment, the agent can judge whether the current action taken is good or bad and thus adjust the learning policy to understand the environment better.
- Discount factor γ. We define the total return in time t as $G_i{}^t$. At timestamp t the agent chooses action a_t to maximize the expectation $G_i{}^t = \Sigma_{t=\tau}^{T} \gamma^{t-\tau} r_i^t$. The discount factor $\gamma \in [0, 1]$ indicates the attention paid to the future returns. Because this is a continuous task, the discount factor prevents the agent from focusing infinitely on the future rewards. Where T represents the total duration of one episode and i is the ith intersection.

In this paper, we take the output of a deep Q-learning network (DQN) [36], namely the action-value function, to approximate the total reward $G_i{}^t$. The loss function of it is

$$\mathcal{L}(\theta) = \mathbb{E}[(r_i^t + \gamma \max_{a'} Q(s_i', a_i'; \theta') - Q(s_i^t, a_i^t; \theta))^2] \tag{2}$$

where a_i', s_i' are the next action and next state, θ is the trainable parameter in MetaSTGAT and $\mathbb{E}$ means expectation value.

4. Method

In this section, we introduce in detail our meta-learning based spatial–temporal graph attention network framework.

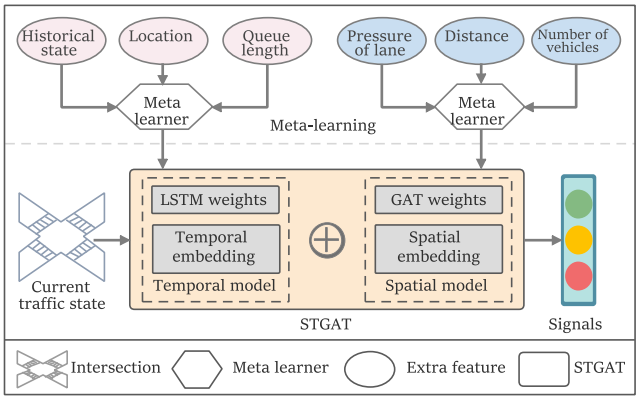


Fig. 3. Architecture of MetaSTGAT.

4.1. Framework

Fig. 3 depicts the architecture of MetaSTGAT for traffic signal control, which consists of a novel designed STGAT and a meta-learning module utilized to generate neural weights. The STGAT module is constructed by the novel multi-head attention mechanism, which has proven very effective in our previous work DynSTGAT [17]. It includes a temporal model and a spatial model, capturing both spatial and temporal features and fully exploiting their joint relations. The meta-learning module focuses on addressing the weight generation problem to adapt the graph network to the changing state of the intersection. We utilize two meta-learners to learn additional intersection features, including historical state, lane pressure, distance, and so much more, as a way to generate new GAT or LSTM weights.

4.2. Spatial–temporal graph attention network

4.2.1. State encoding

At timestamp t, agent gets an observation s_i^t of intersection $\mathcal{I}_i$, which includes the current signal phase and the number of vehicles on each entering lane. We first encode the traffic state s_i^t with a two-layer perceptron, as shown in Eq. (3).

$$\begin{aligned} e_0 &= \mathbf{MLP}(s_i^t) = \sigma(s_i^t W_{11} + b_{11}) \\ e_i &= \mathbf{MLP}(e_0) = \sigma(e_0 W_{21} + b_{21}) \end{aligned} \tag{3}$$

Suppose the dimension of the input state s_i^t is d_0, then the weight $W_{11} \in \mathbb{R}^{d_0 \times d_1}$, and bias $b_{11} \in \mathbb{R}^{d_1}$. Similarly, the following weight $W_{21} \in \mathbb{R}^{d_1 \times d_1}$ and bias $b_{21} \in \mathbb{R}^{d_1}$. Where d_1 is the output dimension of the MLP, so the output e_i is a hidden representation in the d_1-dimensional space. In Eq. (3), σ is the ReLU activation function. Note that e_i is next taken as input to the LSTM to extract the current temporal feature representation x_i, as shown in Fig. 4.

Similarly, we continue to encode the traffic state s_i^t with another MLP of the same configuration to capture the spatial information by GAT, as shown in Eq. (4). Where W_{12}, W_{22} and b_{12}, b_{22} are weights and biases, respectively. e_j is a hidden vector representation in the d_1-dimensional space, as is e_i.

$$\begin{aligned} \hat{e_0} &= \mathbf{MLP}(s_i^t) = \sigma(s_i^t W_{12} + b_{12}) \\ e_j &= \mathbf{MLP}(\hat{e_0}) = \sigma(\hat{e_0} W_{22} + b_{22}) \end{aligned} \tag{4}$$

4.2.2. Current spatial–temporal module

Traffic congestion at one intersection often affects its neighboring intersections during the time of heavy traffic flow. Therefore, it is common to incorporate the impact of neighboring intersections when designing signalization scheme. CoLight [13] implements this idea with a native GAT network, but does not

consider the surrounding intersections in a temporal dimension. Moreover, although many works have borrowed and improved spatial–temporal graph neural networks (STGNN), the spatial and temporal information is simply combined or used separately. To adequately exploit the joint relations of spatial–temporal features, we devise an innovative graph attention network with a GAT to capture spatial information and an LSTM to extract temporal information.

After encoding the traffic state s_i^t, we get two latent representations e_i and e_j, as shown in Fig. 4. Then we operate an LSTM network to capture the current temporal feature x_i with the hidden representation e_i, as defined in Eq. (5), where x_c is the state of the memory gate and x_t is the state of the hidden gate. Furthermore, *cstate* and *hstate* are their initialized states of the LSTM before training.

$$x_i, x_c, x_h = \textbf{LSTM}(e_i, cstate, hstate) \tag{5}$$

$$Q_i = e_j, \quad K_i = x_i, \quad V_i = x_i \tag{6}$$

Next, we take a multi-head attention mechanism, a common approach in machine translation [37], to learn intersection cooperation and capture the spatial–temporal joint information representation. Since the attention mechanism works between adjacent intersections, the target intersection then gets the overall impact of all the neighboring intersections. As formulated in Eq. (6), we regard the hidden traffic representation e_j as the *query* (Q_i), and the current temporal feature x_i as *key* (K_i) and *value* (V_i) in the attention mechanism, respectively. This innovation allows spatial and temporal information to be fully explored, and specifically, the attention mechanism is formulated as:

$$\varphi(Q_i, K_{i,u}) = \frac{Q_i \cdot K_{i,u}}{\sqrt{D_h}} \tag{7}$$

$$\alpha(Q_i, K_{i,u}) = \frac{\exp(\varphi(Q_i, K_{i,u}))}{\sum_{u' \in \mathcal{N}_i} \exp(\varphi(Q_i, K_{i,u'}))} \tag{8}$$

$$Attention^{(1)}(Q_i, K_i, V_i) = \frac{1}{H} \sum_{h=1}^{h=H} \sum_{u} \alpha(Q_i, K_{i,u}) V_{i,u} \tag{9}$$

We first calculate the scaled dot-product $\varphi(Q_i, K_{i,u})$ with *query* and *key*, as shown in Eq. (7). Where D_h denotes the size of the hidden neurons in the attention mechanism and $u \in \mathcal{N}_i$ is the neighboring intersections of intersection $\mathcal{I}_i$. Then, the attention scores $\alpha(Q_i, K_{i,u})$ is gained by the *softmax* function in Eq. (8), which indicates the weight of neighboring intersections influence on the target intersection. Finally, we add up all the impacts of the neighboring intersections to obtain the attention vector, as shown in Eq. (9), where H is the number of multi-head attention. With the above calculation, we obtain the current spatial–temporal representation $z_i^{(CST)}$ of the target intersection $\mathcal{I}_i$ as follows,

$$z_i^{(CST)} = Attention^{(1)}(Q_i, K_i, V_i) \tag{10}$$

Unlike other spatial–temporal graph networks, we carefully design the attention mechanism that tries to fully exploit the joint spatial–temporal relations. And the well-designed mechanism is also applicable to other spatial–temporal problems and scenes.

4.2.3. Current spatial module

In current spatial–temporal module, we take the hidden representation e_i as *query* and the current temporal feature x_i as *key* and *value* to get the current spatial–temporal representation $z_i^{(CST)}$. Similarly, we take e_j as *query*, *key*, and *value* to capture the current spatial representation $z_i^{(CS)}$, as defined by Eq. (11). The calculation operation for the spatial representation $z_i^{(CS)}$ is identical to Eqs. (7)–(9), so we skip and post the final formula directly,

$$\begin{aligned} &Q_i^{(2)} = e_j, \quad K_i^{(2)} = e_j, \quad V_i^{(2)} = e_j \\ &\cdots \\ &z_i^{(CS)} = Attention^{(2)}(Q_i^{(2)}, K_i^{(2)}, V_i^{(2)}) \end{aligned} \tag{11}$$

Usually, the topology of multi-intersection scenarios does not change with time, so we consider the spatial impact at the current time. The current spatial representation $z_i^{(CS)}$ is the sum of the spatial impacts of all neighboring intersections on the target intersection $\mathcal{I}_i$ at the current time.

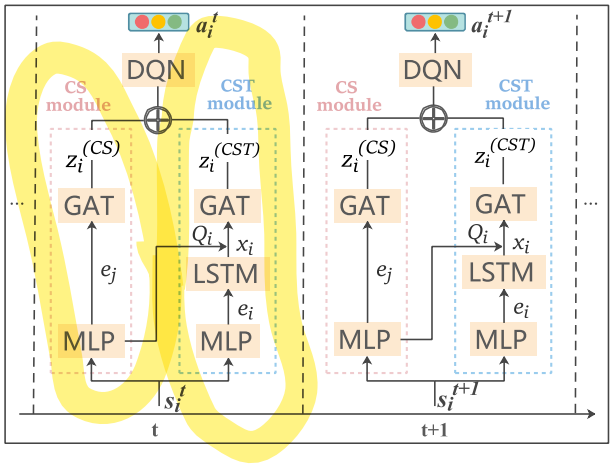


Fig. 4. Architecture of STGAT.

4.2.4. Q-value prediction

In the previous two subsections, we have obtained two spatial–temporal representations $z_i^{(CST)}$ and $z_i^{(CS)}$, and the following deep Q-learning network is required to complete the prediction of these features. DQN will eventually predict the model's Q-value $\widetilde{q}(o_i^t)$ of each feasible action according to Eq. (12). Where **Concat** and **Dense** are the vector concatenate operation and a fully connected network, respectively. Then, the STGAT model chooses the action corresponding to the maximum Q-value for the environment to execute, i.e., to switch the traffic signal.

$$\widetilde{q}(o_i^t) = \textbf{Dense}(\textbf{Concat}(z_i^{(CST)}, z_i^{(CS)})) \tag{12}$$

With the Q-value predicted by the model, we update the pre-defined loss function in Eq. (2) as follows,

$$\mathcal{L}(\theta) = \frac{1}{T} \sum_{t=1}^{T} \sum_{i=1}^{N} (q(o_i^t, a_i^t) - \widetilde{q}(o_i^t, a_i^t; \theta))^2 \tag{13}$$

Where T stands the total simulation time of one episode, N denotes the total number of intersections, and θ is the trainable parameter in the graph network. Its state and the neighboring intersection's states constantly change for each target intersection. Fortunately, our proposed spatial–temporal graph neural network captures the impacts from neighbors in both spatial and temporal dimensions. In summary, the proposed STGAT solves the problem of cooperative control and multi-feature fusion concurrently.

4.3. Meta-learning spatial–temporal graph attention network

Although the proposed STGAT in the previous subsection achieves good results and incorporates both spatial–temporal features, it has an obvious problem. When calculating the attention score, we need to do the scaled dot-product operation in Eq. (7), and the nodes in the default graph are fixed [14],

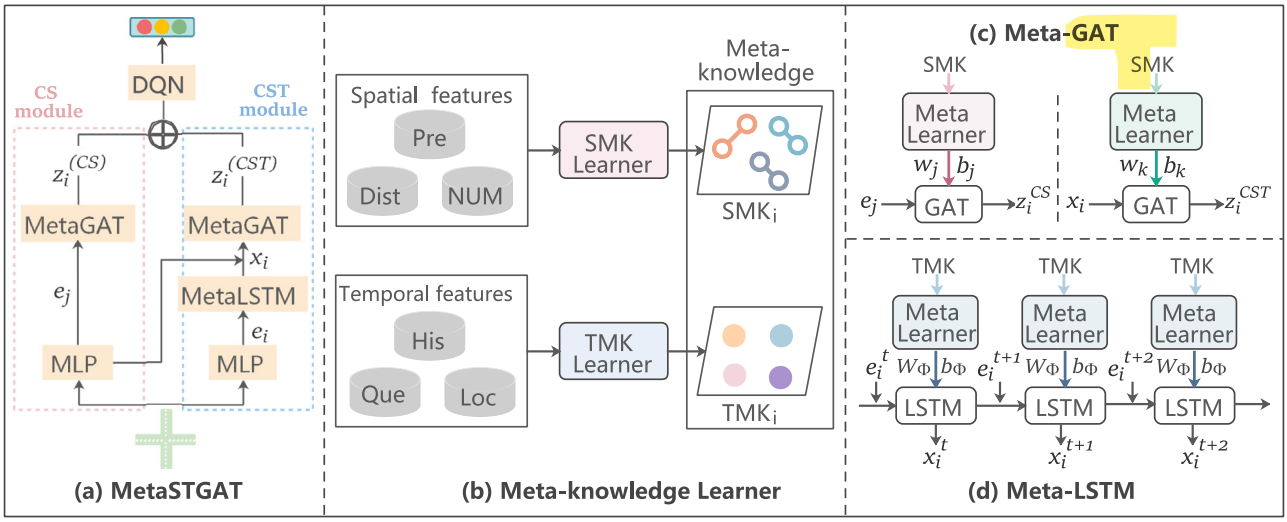


Fig. 5. Overview of the proposed meta-learning based spatial–temporal graph attention network framework-MetaSTGAT.

i.e., the weights of the two nodes are fixed. However, the fact is that the intersection traffic flow is constantly changing, and one attention mechanism cannot be applied to two connected intersections at all times. Thus, we adopt the following meta-learning method to address it and entitle the model MetaSTGAT, as shown in Fig. 5(a).

4.3.1. *Meta-knowledge learner*

We utilize two meta-knowledge learners network to learn the additional spatial features (e.g., distance, lane pressure, and number of vehicles) and temporal features (e.g., historical state, vehicle queue length), as illustrated in Figs. 3 and 5(b). We name the two meta-learners as spatial meta-knowledge learner (SMK-Learner) and temporal meta-knowledge learner (TMK-Learner). Similar to the previous state encoding, the meta-knowledge learner is also a two-layer fully connected network with different initializations. For clarity, we use $\text{SMK}_{(i)}$ and $\text{TMK}_{(i)}$ to represent the learned spatial meta-knowledge and temporal meta-knowledge, respectively. The learned meta-knowledge is then utilized to generate new weights for both the GAT and LSTM networks, as long as the intersection's additional features are obtained, as shown in Fig. 5(c)(d). These new weights reflect the spatial and temporal relations of current traffic conditions. In previous STGAT methods, these changing characteristics are not involved, and access to dynamic spatial–temporal correlations is lost.

4.3.2. *Meta graph attention network*

In a multi-intersection scenario, the spatial correlations are mutually affected and dynamic. Although someone has used GAT to capture these diverse spatial correlations and achieve cooperation, such as CoLight [13], the graph network within this work does not handle the dynamically changing intersection states. This method is inappropriate since it applies the graph network at each timestamp of the intersection, i.e., using the identical attention mechanism.

In this paper, we put forward a meta graph attention network (Meta-GAT) to solve this problem, consisting of a meta-knowledge learner and a GAT network, as illustrated in Fig. 5(c). Specifically, the meta-learner generates weights from the $\text{SMK}_{(i)}$, and then these weights are used in the attention mechanism of GAT. As a result, not only are spatial correlations captured, but they can also vary from different connected intersection pairs. The spatial meta-knowledge consists of the pressure of lanes, the number of vehicles, and the distance of two intersections.

In the proposed attention mechanism, the first step is to do the scaled dot-product operation for two connected intersections as in Eq. (7), then calculate the attention score, i.e., the weights of neighboring intersections act on the target intersection as in Eq. (8), and finally assign these weights to each surrounding intersection before summing as in Eq. (9). Formally, for the current spatial module, we employ a two-layer fully connected network (**MetaDense**) with ReLU activation function to generate new weight W_j and bias b_j, as shown in Eqs. (14), (15), to adapt the attention mechanism to dynamically changing traffic states. Where Q_i equals to $K_{i,u}$ are the hidden traffic representation $e_j \in R^{d_1 \times d_1}$, and $W_j \in \mathbb{R}^{D_h \times d_1}$, D_h is the size of the hidden neurons in the attention mechanism.

$$\begin{aligned} W_j &= \textbf{MetaDense}^{(1)}(\text{SMK}_{(i)}) \in \mathbb{R}^{D_h \times d_1} \\ b_j &= \textbf{MetaDense}^{(1)}(\text{SMK}_{(i)}) \in \mathbb{R} \end{aligned} \tag{14}$$

So far, the weights generated by meta-knowledge $\text{SMK}_{(i)}$ update the spatial attributes of two connected intersections. Then, to model such diverse spatial correlations, we redefine the dot-product operation φ with Eq. (15).

$$\begin{aligned} \varphi(Q_i, K_{i,u}) &= \frac{W_j \cdot (Q_i \cdot K_{i,u}) + b_j}{\sqrt{D_h}} \\ \alpha(Q_i, K_{i,u}) &= \frac{\exp(\varphi)}{\sum_{u' \in \mathcal{N}_i} \exp(\varphi)} \end{aligned} \tag{15}$$

Similarly, for Meta-GAT in the current spatial–temporal module, we update the attention score α with the following two steps in Eqs. (16), (17) to model the dynamic spatial–temporal correlations. The derivation of the remaining part in Meta-GAT is the same as the previous STGAT.

$$\begin{aligned} W_k &= \textbf{MetaDense}^{(2)}(\text{TMK}_{(i)}) \in \mathbb{R}^{D_h \times d_1} \\ b_k &= \textbf{MetaDense}^{(2)}(\text{TMK}_{(i)}) \in \mathbb{R} \end{aligned} \tag{16}$$

$$\begin{aligned} \varphi(Q_i, K_{i,u}) &= \frac{W_k \cdot (Q_i \cdot K_{i,u}) + b_k}{\sqrt{D_h}} \\ \alpha(Q_i, K_{i,u}) &= \frac{\exp(\varphi)}{\sum_{u' \in \mathcal{N}_i} \exp(\varphi)} \end{aligned} \tag{17}$$

Since we extract meta-knowledge from additional features such as the pressure of lanes, the number of vehicles and utilize

such information to generate the weights and biases of graph attention network, Meta-GAT can capture the dynamic spatial–temporal correlations of the intersection.

4.3.3. Meta long short-term memory

To capture the temporal information of the intersection's state, we employ a recurrent neural network (RNN) variant long short-term memory (LSTM) [38] to encode the latent traffic representation e_i. The LSTM avoids the problem of RNN gradient disappearance and learns long-term dependence information, so we take it as an example to illustrate the proposed MetaSTGAT. The LSTM network is defined as,

$$h_t = \textbf{LSTM}\left(e_i^t, h_{t-1} \mid W_\Phi, b_\Phi\right) \tag{18}$$

where e_i^t and h_{t-1} are the input vector and hidden state at timestamp t, respectively. W_Φ is the weight matrix and b_Φ is the bias. The complete derivation of LSTM is as follows: it has three gate units f_t, i_t, o_t, a cell state c_t, and a hidden encoding state h_t. In Eq. (19), σ is *sigmoid* function, ϕ is *tanh* function and $\otimes$ is element-wise multiplication.

$$\begin{aligned} f_t &= \sigma\left(W_f \cdot [h_{t-1}, e_i^t] + b_f\right) \\ i_t &= \sigma\left(W_i \cdot [h_{t-1}, e_i^t] + b_i\right) \\ o_t &= \sigma\left(W_o \cdot [h_{t-1}, e_i^t] + b_o\right) \\ c_t &= f_t \otimes c_{t-1} + i_t \otimes \phi\left(W_s \cdot [h_{t-1}, e_i^t] + b_s\right) \\ h_t &= o_t \otimes \phi(c_t) \end{aligned} \tag{19}$$

Since temporal correlations of the intersection states vary from one signal cycle to another, a simple LSTM in Eq. (19) encoding the current state s_i^t is insufficient to capture diverse temporal correlations. To fully model the dynamic temporal characteristics of the intersection, we borrow a similar technique from Meta-GAT to update the weights and biases of LSTM from temporal embedding, which TMK-Learner learns from temporal features (e.g., vehicle queue length, historical states, and location). We define the meta long short-term memory (Meta-LSTM) as,

$$\begin{aligned} W_\Phi^{(i)} &= \textbf{MetaDense}^{(\mathbf{3})}(\text{TMK}_{(i)}) \in \mathbb{R}^{2D_h \times D_h} \\ b_\Phi^{(i)} &= \textbf{MetaDense}^{(\mathbf{3})}(\text{TMK}_{(i)}) \in \mathbb{R} \end{aligned} \tag{20}$$

$$x_i^t = \textbf{Meta-LSTM}\left(e_i^t, h_{t-1} \mid W_\Phi^{(i)}, b_\Phi^{(i)}\right) \tag{21}$$

Where $W_\Phi^{(i)}$ and $b_\Phi^{(i)}$ are the generated weight and bias of intersection $\mathcal{I}_i$ at timestamp t. As a result, all intersections have their weight and bias, and these values update when the traffic state changes to better deal with dynamic traffic. In the current spatial–temporal module, we take the output x_i^t of Meta-LSTM as the inputs of Meta-GAT, and thus the joint spatial–temporal relations are both captured.

4.4. Algorithm of optimization

Suppose the loss function of MetaSTGAT $\mathcal{L}(\theta)$ is defined in Eq. (13), which measures the prediction Q-value and the ground truth. θ is all trainable parameter in the graph network and deep Q-learning network. Besides, we denote all the meta-knowledge learner trainable parameter as η. For parameter θ in graph network and deep Q-learning network, the gradient of it is $\nabla_\theta \mathcal{L}(\theta)$. Similarly, according to the chain rule we can calculate the gradient of meta-knowledge learner as $\nabla_\eta \mathcal{L}(\theta) = \nabla_\omega \mathcal{L}(\theta) \nabla_\eta \omega$, where ω is the generated parameter in meta-learner. Algorithm 1 summarizes the training process of MetaSTGAT. For each training episode, we first store the training data based on the action predicted by the model (Lines 3–8). Then we optimize the MetaSTGAT through the gradient descent algorithm RMSprop (Lines 9–14).

Algorithm 1: MetaSTGAT for Traffic Signal Control.

Input: Set of target intersection $\mathcal{I}_N$
Output: Optimized parameter θ and η in MetaSTGAT

1: **Initialize:** green time T_g, simulation time T, and all trainable parameters θ and η
2: **for** training episode = 1, 2, ..., N_e **do**
3: $\mathcal{D}_{\text{train}} \leftarrow \emptyset$
4: **for** $t = 1, 2, ..., \frac{T}{T_g}$ **do**
5: generate action set $\mathcal{A}_i^t$ for intersection $\mathcal{I}_i$
6: execute traffic signal with $\mathcal{A}_i^t$
7: store transitions $\{s_t, a_t, r_t, s_{t+1}\}$ into $\mathcal{D}_{\text{train}}$
8: **end for**
9: **for** *each target intersection* $\mathcal{I}_i$ in $\mathcal{I}_N$ **do**
10: sample a batch $\mathcal{D}_{\text{batch}}$ from $\mathcal{D}_{\text{train}}$
11: calculate loss $\mathcal{L}(\theta)$ in Equation (13)
12: $\theta = \theta - \mu \nabla_\theta \mathcal{L}(\theta)$ // μ is learning rate
13: $\eta = \eta - \mu \nabla_\eta \mathcal{L}(\theta)$
14: **end for**
15: **end for**

4.5. Time complexity analysis

We assume that: (1) All intersections can simultaneously predict the corresponding Q-value by MetaSTGAT. (2) Multi-head attention calculation is as fast as a single attention head. (3) The embedding operations with weights (such as W_{11}, W_{21} and W_k) can also be executed at the same time. (4) We only consider the multiplication operations since the addition operation is not particularly important. Suppose the input state dimension is k, the signal phase space is p, the number of hidden layers is L, and the hidden size of each layer is m. The time complexity in component of MetaSTGAT is: (1) state encoding: 2 km; (2) meta graph attention network: $(m^2 + m^2)L$; (3) meta long short-term memory: $4(km + m^2 + m)$; (4) q-value prediction: mp; Thus, the time complexity is $O(m^2(2L + 4) + m(6k + p + 4))$, and it is approximately equal to $O(m^2 L)$.

5. Experiments

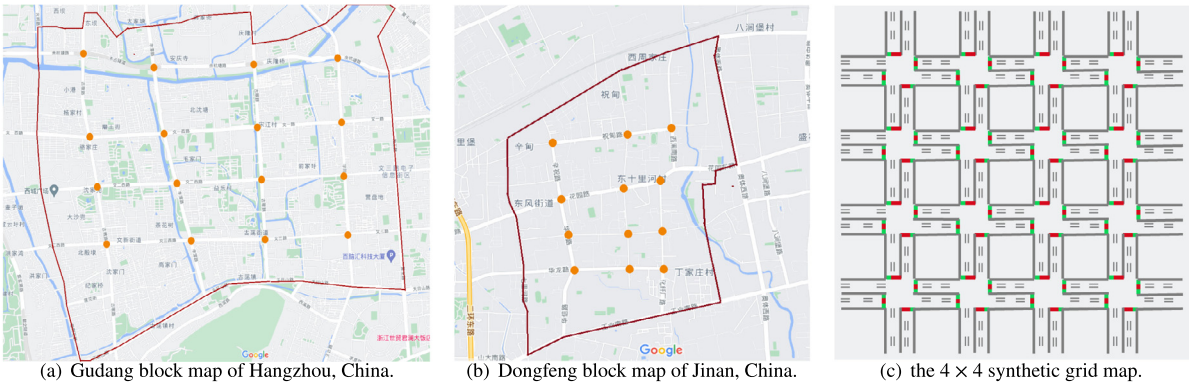
In this section, we perform simulation experiments on the synthetic dataset and real-world dataset to evaluate our model MetaSTGAT and other advanced methods, particularly to respond to the following questions.

- **RQ1**. Can MetaSTGAT outperform STGAT or state-of-the-art methods?
- **RQ2**. How much improvement does the meta-learning component bring?
- **RQ3**. How is the convergence and stability of MetaSTGAT in the training stage compared with other methods?
- **RQ4**. How do the settings of MetaSTGAT (e.g., numbers of neighbor intersections, numbers of attention heads) affect the results?

5.1. Settings

We experiment with the traffic simulator CityFlow,[2] which provides rich interfaces and functions and can implement city-level traffic signal control [39]. We obtain the intersection's state, such as the number of vehicles, location, and signal phase, by calling API. Then the simulator executes traffic signals after the

[2] https://cityflow-project.github.io.



(a) Gudang block map of Hangzhou, China. (b) Dongfeng block map of Jinan, China. (c) the 4 × 4 synthetic grid map.

Fig. 6. Road structures of the real-world datasets (a) (b) and grid map of the synthetic dataset (c). We choose to model the red polygon area on the real-world road networks, and the orange dot is the intersection set with signal lights. The other two datasets have bi-directional traffic flow except for the Hangzhou dataset, which has uni-& bi-directional mixed traffic flow.

agent takes one signal phase. According to the default setting in work [11], the green signal executes ten seconds, and there will be a three-second yellow time and a two-second red time immediately after the end of the green signal to allow the remaining vehicles to cross the intersection.

Experiment settings: The simulation duration of the synthetic dataset and real-world dataset are 1800 s and 3600 s per round, respectively. We take an Root Mean Square propagation (RMSprop) optimizer to train 200 total rounds. To guarantee the diversity of training samples, we run three processes in parallel for each dataset when training the model. **Agent settings**: Training batch size is set to 20, the discount factor in reinforcement learning algorithm is $\gamma = 0.85$, and each episode is trained 100 times. The sampling size is 1000 and the experience replay buffer size is 10 000. **Traffic settings**: The green light time is set at 10 s, the yellow light time is set at 3 s, and the red light time at 2 s. Each target intersection's neighbors are 4.

5.2. Datasets

We experiment on a 4 × 4 grid-like synthetic dataset and two real-world datasets of Hangzhou and Jinan. For each vehicle in the these datasets, it is represented as ($\mathbf{o}$, $\mathbf{t}$, $\mathbf{d}$), where $\mathbf{o}$ is starting location, $\mathbf{t}$ stands departure time, and $\mathbf{d}$ stands the destination location.

For each signalized intersection in the synthetic dataset, the road structure has four entering and exiting ways of 300 m. All traffic data has been converted into *JSON* format, and we run the real-world or synthetic data in the simulator by loading the road network files and traffic flow files. The dataset is described in detail as follows:

- **Synthetic dataset:** the 4 × 4 grid map is illustrated in Fig. 6(c). In this road network, each is a standard intersection of 300 meters long, as shown in Fig. 1. Depending on the traffic flow settings, we have the following four configurations of different traffic demands: the average arrival rate of vehicles is 0.388 vehicle/s and 0.416 vehicle/s, and the traffic flow of each arrival rate has two different variances 0.3 (Flat) and 0.6 (Peak), as reported in Table 1. This dataset is synthesized based on the statistical analysis of real-world traffic patterns in Jinan and Hangzhou. According to the statistical analysis of taxi numbers on the real-world dataset, we assign the turn probabilities at intersections to 10% (left-turning), 60% (straight), and 30% (right-turning).
- **Real-world datasets:** These two traffic datasets were collected by surveillance cameras and underwent a series of pre-processing. The road structures are shown in Fig. 6(a)(b), and the number of intersections and the statistical arrival rate are reported in Table 2.

  $D_{Hangzhou}$: We select 16 adjacent intersections in Gudang block, Hangzhou, China, as the traffic scenario, as shown in Fig. 6(a). The cameras have recorded related information in this dataset, including the camera ID, timestamp, and vehicle speed. After the statistical analysis and preliminary processing of these data, the trajectory of vehicles crossing the intersection is preserved. We calculate the number of vehicles passing through the intersection and take it as the traffic volume.

  D_{Jinan}: This traffic dataset recording method and preprocessing are similar to $D_{Hangzhou}$, but it incorporates 12 adjacent intersections in Dongfeng block, Jinan, China, as illustrated in Fig. 6(b).

Table 1
Four configurations with different traffic demand of the synthetic dataset.

Config	Demand pattern	Arrival rate (vehicles/s)
1	Flat	0.388
2	Peak	
3	Flat	0.416
4	Peak	

Table 2
Traffic volume statistics of the real-world dataset.

Dataset	Intersections	Arrival rate (vehicles/300 s)			
		Mean	Std	Max	Min
$D_{Hangzhou}$	16	526.63	86.70	676	256
D_{Jinan}	12	250.70	38.21	335	208

5.3. Compared methods

We compare the proposed MetaSTGAT with the traditional and RL-based graph methods to verify its effectiveness and efficiency. To make a fair comparison, we train each model from scratch and debug the parameters, taking the average value of the last ten tests as the final result.

- **FixedTime** [1]. This method sets the signal time as a fixed value, which is mainly deployed for the control of steady traffic flow.
- **NeighborRL** [8]. A multi-agent method concatenates their neighboring intersections' states to achieve cooperative control, and all agents share identical parameters.

Table 3
Performance comparison on four traffic demand configurations of the synthetic dataset. Shorter travel times mean better models, reported in seconds. The higher the throughput, the better. The following table is the same.

Method	Travel time (s)				Throughput (vehicle)			
	Config 1	Config 2	Config 3	Config 4	Config 1	Config 2	Config 3	Config 4
FixedTime [1]	573.1	564.0	536.0	563.1	3555	3477	3898	3556
NeighborRL [8]	690.9	687.3	781.2	791.4	3504	3255	2863	2537
CGRL [12]	735.4	758.6	771.1	721.4	3122	2792	2962	2991
GCN [40]	516.7	523.8	646.2	585.9	4275	4151	3660	3695
CoLight [13]	515.9	517.0	553.4	524.3	4206	3480	3853	3891
DynSTGAT [17]	471.9	445.6	519.0	500.1	5003	4312	4852	4265
STGAT	513.4	501.7	539.0	516.8	4823	4152	4577	4218
MetaSTGAT	**452.8**	**413.2**	**476.8**	**467.1**	**5026**	**4454**	**4997**	**4460**

Table 4
Performance results reported on the real-world datasets.

Method	Travel time (s)		Throughput (vehicle)	
	Hangzhou	Jinan	Hangzhou	Jinan
FixedTime [1]	728.8	869.9	–*	–*
NeighborRL [8]	1053.5	1168.3	–*	–*
CGRL [12]	1582.3	1210.7	–*	–*
GCN [40]	768.4	625.7	–*	–*
CoLight [13]	303.5	294.0	2974	5513
DynSTGAT [17]	286.5	287.6	2974	5572
STGAT	289.1	305.8	2974	5420
MetaSTGAT	**278.5**	**268.3**	**2974**	**5717**

*Denotes that we do not conduct experiments on this dataset since we want to compare out model with the graph-based or spatial–temporal based methods.

- **CGRL** [12]. A multi-agent method coordinates traffic lights with modeling joint-action. Specifically, policy transfer and coordination algorithm are adopted.
- **GCN** [40]. This method is implemented based on a graph convolution network (GCN), and it combines the neighboring traffic conditions for collaborative control.
- **CoLight** [13]. A large-scale and network-level method with a Graph Attention Network (GAT) to facilitate communication and cooperation at the intersections.
- **DynSTGAT** [17]. Our previously published method is still a static graph network based on STGAT and it contains a temporal module captured with TCN [41] and LSTM and a spatial module acquired with GAT.
- **STGAT**. Our baseline model, with a novel attention mechanism, employs a graph attention network to extract spatial features and an LSTM network to acquire temporal features, which are then adequately fused.
- **MetaSTGAT**. The meta-learning approach is finely developed to improve the basic model STGAT for adapting to dynamic traffic flow.

5.4. Evaluation metrics

The two most effective and representative evaluation metrics are adopted to evaluate different methods [3].

- **Travel time**. It represents the average time in this round that all vehicles spend on the road networks, namely the time (in seconds) from the place of departure to the destination. Shorter travel time means better control of the model.
- **Throughput**. This metric stands for the number of vehicles that have completed the trip in this round of simulation. Greater throughput means more vehicles pass through the intersection and arrive at their destination, implying a better signal control scheme.

5.5. Performance analysis

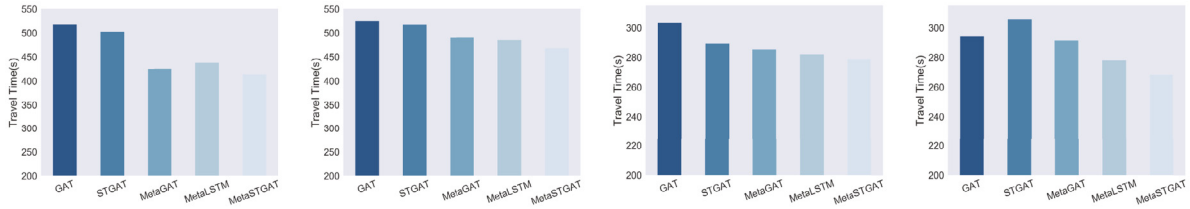
In this chapter, we first evaluate the performance of MetaSTGAT in travel time and throughput and compare it with other traditional and RL-based graph methods. We then evaluate the improvements brought about by meta-learning and conduct case studies.

5.5.1. Overall evaluation

We present all compared methods' results in Table 3 and Table 4, respectively. These methods to be compared include a traditional method, non-graph network methods, and graph networks such as GCN and GAT. We get the following intuitive observations and numerical analyses:

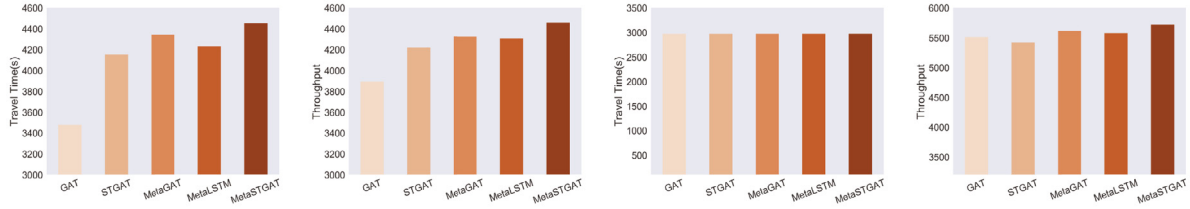
In Table 3, we observe that MetaSTGAT beats all compared methods under four different configurations, achieving the least travel time and the largest throughput. Based on this, we conclude that rephrase question 1 (**RQ1**) has been confirmed. Specifically, compared with the baseline method STGAT, MetaSTGAT achieves a maximum improvement of 17.64% in travel time under Config-2 and 11.84% in throughput under Config 3, respectively. However, our previous state-of-the-art model DynSTGAT only outperforms STGAT with a maximum improvement of 11.18% and 6.01% in Config-2 and Config 3, respectively. Although it works well, it is not as good as MetaSTGAT when dealing with heavy or dynamic traffic flow. For example, the traffic volume under Config-2 and Config-4 are light and heavy, respectively, and all have a relatively large variance of 0.6, but MetaSTGAT is significantly better than STGAT and more stable. From Config-2 to Config-4, MetaSTGAT drops from 17.64% to 9.62%, while DynSTGAT drops from 11.18% to 3.23%. Compared with the graph-level method CoLight, MetaSTGAT fully combines the spatial and temporal joint information with a novel attention mechanism proven effective. Moreover, it is not difficult to observe that the performance of reinforcement learning methods are better than FixedTime. The reason is that RL methods learn the current traffic state and directly optimize the intersection pressure in time to adjust the signal light switching policy.

Table 4 reports the travel time and throughput on real-world datasets Hangzhou and Jinan, which once again verifies the feasibility of **RQ1**. Obviously, MetaSTGAT is better than the rest method, and it is 8.24% and 8.74% higher than CoLight in travel time, respectively, on Hangzhou and Jinan datasets. In the same situation, STGAT has only 4.74% and −4.0%. The four graph-based methods have the same throughput of 2974 on the Hangzhou dataset because all vehicles complete their trips within 3600 s of simulation time. However, the simulation time and the total number of vehicles for the Jinan dataset are 3600 s and 6296, respectively. Furthermore, we observe that common RL methods such as NeighborRL and CGRL are not as good as FixedTime. These two methods expand the observation scope by merging the states of connected intersections while ignoring the spatial correlation.



(a) Travel time on Config-2 (b) Travel time on Config-4 (c) Travel time on Hangzhou (d) Travel time on Jinan

Fig. 7. Travel time evaluated under different models.



(a) Throughput on Config-2 (b) Throughput on Config-4 (c) Throughput on Hangzhou (d) Throughput on Jinan

Fig. 8. Throughput evaluated under different models.

However, the graph-based approach changes this situation and outperforms the first three methods.

When comparing the performance of MetaSTGAT with the FixedTime, we discover that the gap in travel times becomes more notable when the dataset contrasts from synthetic traffic (18.95% on average) to real-world traffic (25.96% on average). The reason is that vehicle's arrival rate of the real-world dataset is twice of the synthetic dataset, which means that the real-world traffic flow is bigger. At the same time, traditional methods cannot directly learn from the environment and waste much time.

Based on the above observations and analyses, it is not hard to conclude that the proposed method is better than other methods with comparisons. Although there is much room for improvement compared to peer graph network methods, this is only the results on limited datasets are enough to illustrate its advantages. Thus, our method is straightforward to deploy to larger-scale traffic scenarios, such as urban intersections with thousands of lights.

5.5.2. Evaluation of meta-learning

Comparative experiments are performed on these different variants to present the effectiveness brought by each meta-learning component. These models include GAT (only a GAT), STGAT (LSTM, GAT), MetaGAT(LSTM, Meta-GAT), MetaLSTM (GAT, Meta-LSTM) and MetaSTGAT (Meta-LSTM, Meta-GAT). The remark networks in each model are used to handle spatial or temporal features, respectively.

Figs. 7 and 8 show that the dark blue histogram evaluates travel time, and the orange histogram evaluates its corresponding throughput. These results are tested on Config-2, Config-4, and Hangzhou and Jinan datasets. The performance of the other four variable GAT models consistently outperforms single GAT in travel time and throughput. Model MetaSTGAT, which combines both Meta-LSTM and Meta-GAT networks, achieves the best results. Therefore, the meta-learning based method improves the performance of traditional networks. Although the improvements are not the same, it is undeniable that it has brought changes to the previous network, confirming the hypothesis of **RQ2**. Specifically, MetaSTGAT improves travel time the most with Config-2 and Hangzhou datasets, followed by Config-4 and Jinan. However, the order of throughput improvement is Config-2 and Config-4, followed by Jinan and Hangzhou.

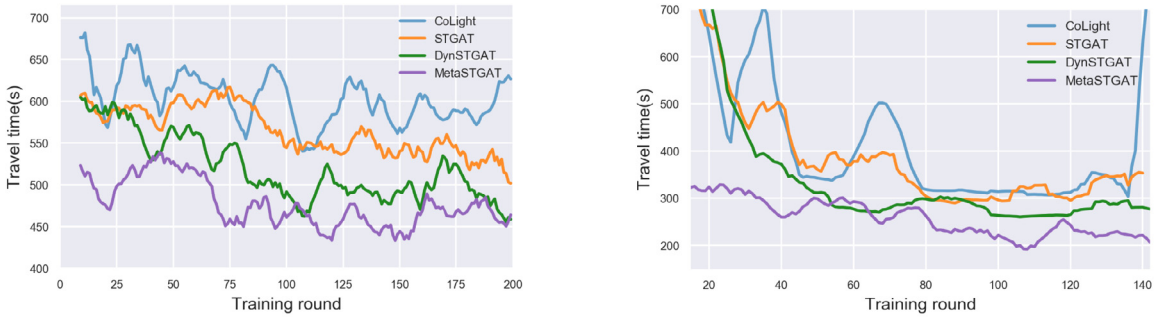
5.5.3. Study of MetaSTGAT

- Convergence Speed and Stability. As illustrated in Fig. 9, we plot MetaSTGAT's performance curve of travel time on synthetic and Hangzhou datasets and compare it with three graph network methods. The training process shows that MetaSTGAT significantly outperforms the other three methods in convergence speed, stability, and training results. Both figures illustrate that our method fluctuates less during training, and other methods such as STGAT only perform relatively stable on the Hangzhou dataset. So, it is confirmed that our proposed method in **RQ3** has certain advantages.
- Effect of Neighbor Number. Fig. 10(a) shows the results of MetaSTGAT training 100 and 200 episodes, respectively, with different numbers of neighbors. When the number of neighbors reaches 4 or 5, the model brings the least travel time. A more significant numbers of neighbors makes the model require more training rounds. Therefore, taking the state influence of the nearest four intersections into account is sufficient to ensure the performance of traffic signal control.
- Effect of Attention Head Number. We test and validate the sensitivity of two models, STGAT and MetaSTGAT, to the number of attention heads on the Hangzhou dataset. Fig. 10(b) shows that both models achieve the shortest travel time with four attention heads, while STGAT is more sensitive to the number of attention heads. When the numbers of attention heads H is greater than 5, the advantage of multi-head attention becomes less noticeable.

The above two paragraphs analyze the impact of different numbers of neighbors and numbers of attention heads on model performance. Thus, our proposed method outperforms other start-of-the-art methods not only by choice of these two parameters but also by the novelty of the method. Specifically, we adopt a spatial–temporal graph attention network from the perspective of cooperative signal control and then improve the graph attention network based on meta-learning according to the real-world problem of dynamic traffic flow. We have also drawn the same conclusions on the synthetic datasets, which have answered **RQ4**.

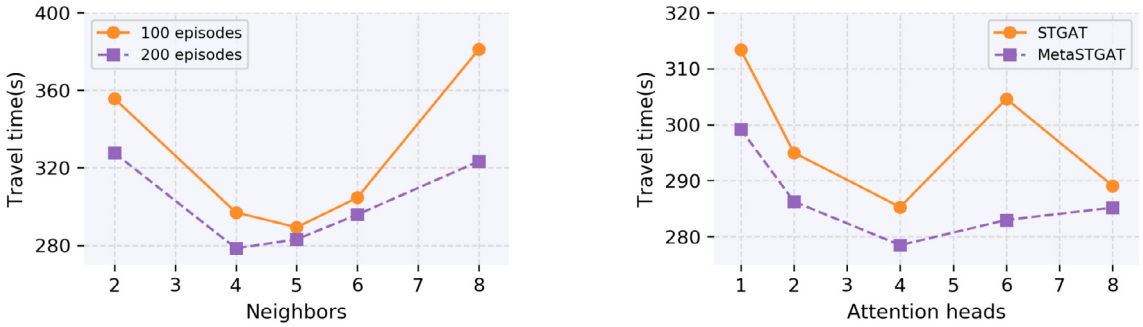
6. Conclusion

In this work, we design an explicit reinforcement learning based and meta-learning based spatial–temporal graph attention network (MetaSTGAT) framework to resolve two challenges in traffic signal control: (1) How to competently combine the



(a) Synthetic dataset under Config-2. (b) Real-world dataset of Hangzhou.

Fig. 9. Convergence curves comparison during training on two different datasets.



(a) Effect of the neighbors (b) Effect of the attention head.

Fig. 10. Performance comparison on $D_{Hangzhou}$ with different numbers of neighbors and attention heads.

joint spatial–temporal attribute characteristics of intersections with graph network. (2) How to improve the graph network through meta-learning to adapt it to the dynamic traffic flow. Specifically, we first put forward a novel attention mechanism based spatial–temporal model STGAT for traffic signal control, which differs from other methods in the adequate exploiting of spatial–temporal information. Moreover, we use meta-learning to improve STGAT through weight update or generation to tackle the dynamic node changes, i.e., changing traffic flow at the intersection. Extensive simulation experiments confirm the notable performance of MetaSTGAT compared with other methods. In the future, we will expand our research ideas to larger-scale urban traffic signal control and explore the feasibility of this approach in other urban computing tasks such as traffic prediction.

CRediT authorship contribution statement

Min Wang: Conceptualization, Methodology, Data curation, Writing – original draft. **Libing Wu:** Supervision. **Man Li:** Visualization, Investigation. **Dan Wu:** Software, Validation. **Xiaochuan Shi:** Software, Validation. **Chao Ma:** Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported by the National Natural Science Foundation of China (No. U20A20177, 61772377), the National Key Research and Development Program of China (No. 2021YFB3101104), the Opening Foundation of the State Key Laboratory of Integrated Services Networks under Grant ISN21-10, and the Key R&D plan of Hubei Province, China (No. 2021BAA025).

References

[1] L. Singh, S. Tripathi, H. Arora, Time optimization for traffic signal control using genetic algorithm, Int. J. Recent Trends Eng. 2 (4) (2009).
[2] S.B. Cools, C. Gershenson, B. D'Hooghe, Self-organizing traffic lights: A realistic simulation, in: Advances in Applied Self-Organizing Systems, Springer, 2013, pp. 45–55.
[3] H. Wei, G. Zheng, V. Gayah, Z. Li, A survey on traffic signal control methods, 2019, arXiv preprint arXiv:1904.08117.
[4] K.L.A. Yau, J. Qadir, H.L. Khoo, M.H. Ling, P. Komisarczuk, A survey on reinforcement learning models and algorithms for traffic signal control, ACM Comput. Surv. 50 (2017) 1–38.
[5] C. Chen, H. Wei, N. Xu, G. Zheng, M. Yang, Y. Xiong, K. Xu, Z. Li, Toward a thousand lights: Decentralized deep reinforcement learning for large-scale traffic signal control, in: Proceedings of the 34th AAAI Conference on Artificial Intelligence, 2020, pp. 3414–3421.
[6] X. Zang, H. Yao, G. Zheng, N. Xu, K. Xu, Z. Li, Metalight: Value-based meta-reinforcement learning for traffic signal control, in: Proceedings of the 34th AAAI Conference on Artificial Intelligence, 2020, pp. 1153–1160.
[7] G. Zheng, Y. Xiong, X. Zang, J. Feng, H. Wei, H. Zhang, Y. Li, K. Xu, Z. Li, Learning phase competition for traffic signal control, in: Proceedings of the 28th ACM International Conference on Information & Knowledge Management, 2019, pp. 1963–1972.
[8] I. Arel, C. Liu, T. Urbanik, A.G. Kohls, Reinforcement learning-based multi-agent system for network traffic signal control, IET Intell. Transp. Syst. 4 (2010) 128–135.
[9] B.C. Da Silva, E.W. Basso, F.S. Perotto, A.L. C. Bazzan, P.M. Engel, Improving reinforcement learning with context detection, in: Proceedings of the 5th International Joint Conference on Autonomous Agents and Multiagent Systems, 2006, pp. 810–812.
[10] S. El-Tantawy, B. Abdulhai, H. Abdelgawad, Multiagent reinforcement learning for integrated network of adaptive traffic signal controllers: methodology and large-scale application on downtown toronto, IEEE Trans. Intell. Transp. Syst. 14 (2013) 1140–1150.
[11] T. Chu, J. Wang, L. Codecà, Z. Li, Multi-agent deep reinforcement learning for large-scale traffic signal control, IEEE Trans. Intell. Transp. Syst. 21 (2019) 1086–1095.
[12] E. Van der Pol, F.A. Oliehoek, Coordinated deep reinforcement learners for traffic light control, in: NIPS'16 Workshop on Learning, Inference and Control of Multi-Agent Systems, 2016.

[13] H. Wei, N. Xu, H. Zhang, G. Zheng, X. Zang, C. Chen, W. Zhang, Y. Zhu, K. Xu, Z. Li, Colight: Learning network-level cooperation for traffic signal control, in: Proceedings of the 28th ACM International Conference on Information & Knowledge Management, 2019, pp. 1913–1922.
[14] G. Veličković, A. Casanova, A. Romero, P. Lio, Y. Bengio, Graph attention networks, 2017, arXiv preprint arXiv:1710.10903.
[15] X. Fang, J. Huang, F. Wang, L. Zeng, H. Liang, H. Wang, Constgat: Contextual spatial–temporal graph attention network for travel time estimation at baidu maps, in: Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, 2020, pp. 2697–2705.
[16] X. Zhang, C. Huang, Y. Xu, L. Xia, Spatial–temporal convolutional graph attention networks for citywide traffic flow forecasting, in: Proceedings of the 29th ACM International Conference on Information & Knowledge Management, 2020, pp. 1853–1862.
[17] L. Wu, M. Wang, D. Wu, J. Wu, Dynstgat: Dynamic spatial–temporal graph attention network for traffic signal control, in: Proceedings of the 30th ACM International Conference on Information & Knowledge Management, 2021, pp. 2150–2159.
[18] T.N. Kipf, M. Welling, Semi-supervised classification with graph convolutional networks, 2016, arXiv preprint arXiv:1609.02907.
[19] Z. Peng, W. Huang, Q. Luo, Y. Rong, T. Xu, J. Huang, Graph representation learning via graphical mutual information maximization, in: Proceedings of the Web Conference, 2020, pp. 259–270.
[20] S. Zhang, H. Yin, T. Chen, Q.V.N. Hung, Z. Huang, L. Cui, Gcn-based user representation learning for unifying robust recommendation and fraudster detection, in: Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval, 2020, pp. 689–698.
[21] E. Rossi, B. Chamberlain, F. Frasca, D. Eynard, F. Monti, M. Bronstein, Temporal graph networks for deep learning on dynamic graphs, 2020, arXiv preprint arXiv:2006.10637.
[22] A. Sankar, Y. Wu, W. Gou, H. Yang, Dynamic graph representation learning via self-attention networks, 2018, arXiv preprint arXiv:1812.09430.
[23] R. Trivedi, M. Farajtabar, P. Biswal, H. Zha, Dyrep: Learning representations over dynamic graphs, in: International Conference on Learning Representations, 2019.
[24] Z. Pan, W. Zhang, Y. Liang, W. Zhang, Y. Yu, J. Zhang, Y. Zheng, Spatio-temporal meta learning for urban traffic prediction, IEEE Trans. Knowl. Data Eng. 34 (2020) 1462–1476.
[25] Z. Pan, Y. Liang, W. Wang, Y. Yu, Y. Zheng, J. Zhang, Urban traffic prediction from spatio-temporal data using deep meta learning, in: Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, 2019, pp. 1720–1730.
[26] F. Liu, Z. Cheng, Z. Zhu, L. Nie, Interest-aware message-passing gcn for recommendation, in: Proceedings of the Web Conference, 2021, pp. 1296–1305.
[27] R. Huang, C. Huang, G. Liu, W. Kong, Lsgcn: Long short-term traffic prediction with graph convolutional networks, in: Proceedings of the International Joint Conference on Artificial Intelligence, IJCAI, 2020, pp. 2355–2361.
[28] L. Mengzhang, Z. Zhanxing, Spatial–temporal fusion graph neural networks for traffic flow forecasting, 2020, arXiv preprint arXiv:2012.09641.
[29] X. Hu, C. Zhao, G. Wang, A traffic light dynamic control algorithm with deep reinforcement learning based on gnn prediction, 2020, arXiv preprint arXiv:2009.14627.
[30] M. Huisman, J.N. Van Rijn, A. Plaat, A survey of deep meta-learning, Artif. Intell. Rev. 54 (2021) 4483–4541.
[31] L. Bertinetto, J.a. F. Henriques, J. Valmadre, P. Torr, A. Vedaldi, Learning feed-forward one-shot learners, in: Proceedings of the 30th International Conference on Neural Information Processing Systems, 2016, pp. 523–531.
[32] J. Chen, X. Qiu, P. Liu, X. Huang, Meta multi-task learning for sequence modeling, in: Proceedings of the 32nd AAAI Conference on Artificial Intelligence, 2018, pp. 5070–5077.
[33] C. Zhang, M. Ren, R. Urtasun, Graph hypernetworks for neural architecture search, 2018, arXiv preprint arXiv:1810.05749.
[34] V. Garcia, J. Bruna, Few-shot learning with graph neural networks, 2018, arXiv preprint arXiv:1711.04043.
[35] L. Liu, T. Zhou, J. Long, C. Zhang, Learning to propagate for graph meta-learning, in: Proceedings of the 33th International Conference on Neural Information Processing Systems, 2019, pp. 1039–1050.
[36] M. Roderick, J. MacGlashan, S. Tellex, Implementing the deep q-network, 2017, arXiv preprint arXiv:1711.07478.
[37] D. Bahdanau, K. Cho, Y. Bengio, Neural machine translation by jointly learning to align and translate, 2014, arXiv preprint arXiv:1409.0473.
[38] F.A. Gers, J. Schmidhuber, F. Cummins, Learning to forget: Continual prediction with lstm, Neural Comput. 12 (2000) 2451–2471.
[39] H. Zhang, S. Feng, C. Liu, Y. Ding, Y. Zhu, Z. Zhou, W. Zhang, Y. Yu, H. Jin, Z. Li, Cityflow: A multi-agent reinforcement learning environment for large scale city traffic scenario, in: Proceedings of the World Wide Web Conference, ACM, 2019, pp. 3620–3624.
[40] T. Nishi, K. Otaki, K. Hayakawa, T. Yoshimura, Traffic signal control based on reinforcement learning with graph convolutional neural nets, in: Proceedings of the International Conference on Intelligent Transportation Systems, IEEE, 2018, pp. 877–883.
[41] S. Bai, J.Z. Kolter, V. Koltun, An empirical evaluation of generic convolutional and recurrent networks for sequence modeling, 2018, arXiv preprint arXiv:1803.01271.